

Customer Management in CRM: Optimizing Portfolio & Customer Strategy.

1. Situation

An e-commerce platform is preparing for an End-of-Year Appreciation program. The gift budget is limited, so the customer base needs to be segmented into different groups based on the total profit they bring in, to offer appropriate gifts.

2. Problem Statement

- Classify customers using ABC analysis. What percentage of the total customer base does Group A represent? (Does the Pareto principle apply?)
- If the company only has 500 special gifts, does the list of Group A customers exceed this number?
- By further analyzing customer income and occupation information, what marketing campaign suggestions can be made to attract more new Group A customers?

3. Data Used

- EcomSales.csv: Stores sales information.
- Customer.csv: Stores customer information.

I. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as dt

pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.expand_frame_repr', None)
```

II. Load Data and Clean

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

2.1. Load Data

```
df_sales = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/EcomSales.csv')
df_customers = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Customer.csv')
```

```
df_sales.head()
```

```
df_customers.head()
```

```
df_products
```

```
df_region.head()
```

2.2. Data Inspection

```
def check_df(df):

    print('=' * 50)
    print('THÔNG TIN DATAFRAME')
    print('=' * 50)
```

```
df.info()

print('=' * 50)
print('KIỂM TRA GIÁ TRỊ NULL')
print('=' * 50)
print(df.isna().sum())

print('=' * 50)
print('KIỂM TRA DỮ LIỆU TRÙNG LẶP')
print('=' * 50)
print(f"Số lượng trùng lặp: {df.duplicated().sum()}")

print('=' * 50)
print('THỐNG KÊ MÔ TẢ')
print('=' * 50)
print(df.describe().T)
```

2.2.1. Inspect df_sales

```
check_df(df_sales)
```

```
sns.boxplot(data = df_sales, x = 'Sales')
```

```
sns.boxplot(data = df_sales, x = 'Quantity')
```

```
sns.boxplot(data = df_sales, x = 'Profit')
```

Note: Sales, Quantity, Profit columns have outliers.

2.2.2. Inspect df_customers

```
check_df(df_customers)
```

```
sns.boxplot(data = df_customers, x = 'AnnualIncome')
```

Note: Gender column has null values and AnnualIncome has outliers.

2.3. Preprocessing and Data Cleaning

- Change data types
- Correct typos
- Handle duplicate data
- Handle null values
- Handle outliers
- ...

2.3.1. Handle null values

```
df_customers['Gender'].fillna('Undefined', inplace = True)
df_customers[df_customers['Gender'] == 'Undefined'].head()
```

2.3.2. Handle data entry errors

```
import re

# Data format for ProductCode column in df_products (1 uppercase letter followed by 6 digits, e.g., P000001...)
pattern = r'^[A-Z]\d{6}$'

# Find data that does not conform to the above format
non_conforming_product_codes = df_sales[~df_sales['ProductCode'].str.fullmatch(pattern, na=False)]

print(non_conforming_product_codes)
```

```
# Data format for RegionCode column in df_region (1 uppercase letter followed by 4 digits, e.g., R0001...)
pattern = r'^[A-Z]\d{4}$'

# Find data that does not conform to the above format
non_conforming_region_codes = df_sales[~df_sales['RegionCode'].str.fullmatch(pattern, na=False)]

print(non_conforming_region_codes)
```

For data with different code formats than in the dimension table, it is necessary to check with upstream sources or stakeholders before modifying these unusual product and region codes.

2.3.3. Handle outliers in df_sales

```
def outlier_check(df,column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]

    count_outliers = len(outliers)
    outlier_pct = round((count_outliers / len(df)) * 100, 2)

    return {
        'Q1': round(float(Q1),2),
        'Q3': round(float(Q3),2),
        'IQR': round(float(IQR),2),
        'lower_bound': round(float(lower_bound),2),
        'upper_bound': round(float(upper_bound),2),
        'count_outliers': count_outliers,
        'outlier_pct': outlier_pct
    }
```

```
outlier_check(df_sales, 'Sales')
```

```
df_sales[df_sales['Sales'] > 300].sort_values('Sales', ascending = False)
```

```
# Check the number of losing orders
len(df_sales[df_sales['Profit'] < 0])
```

Orders with sales identified as outliers are likely from customers belonging to Groups A and B.

Also, large sales mean a large quantity of products in a single order (in the `Quantity` column), and `Profit` also fluctuates strongly, leading to these becoming outliers.

2.3.4. Handle outliers in df_customers

```
outlier_check(df_customers, 'AnnualIncome')
```

1.64% of the data identified as outliers could be potential Group A customers.

III. Problem Analysis

```
# Connect data for analysis
df_forABC = pd.merge(df_sales, df_customers, on = 'CustomerID', how = 'left')

df_forABC.head()
```

```
# 1. Group and calculate profit by customer
customer_profit = df_forABC.groupby(by = 'CustomerID', as_index = False).agg({'Profit': 'sum'})
customer_profit.head()
```

```
# 2. Sort customers by highest profit
customer_profit = customer_profit.sort_values('Profit', ascending = False).reset_index(drop = True)
customer_profit
```

```
# 3. Calculate cumulative profit percentage
customer_profit['cum_profit'] = customer_profit['Profit'].cumsum()
customer_profit['cum_pct'] = (customer_profit['cum_profit'] / customer_profit['Profit'].sum()) * 100
customer_profit
```

```
# 4. Assign ABC labels to each product
condition = [
    (customer_profit['cum_pct'] <= 80),
    (customer_profit['cum_pct'] > 80) & (customer_profit['cum_pct'] < 95),
    (customer_profit['cum_pct'] >= 95)
]

choices = ['A', 'B', 'C']

customer_profit['ABC_rank'] = np.select(condition, choices, default= 'C')

customer_profit
```

```
# Merge with demographic information (Income, Occupation)
customer_analysis = pd.merge(customer_profit, df_customers[['CustomerID', 'AnnualIncome', 'Occupation']], on = 'CustomerID', how
customer_analysis
```

```
# Prepare aggregated data for charts
abc_summary = customer_analysis.groupby(by= 'ABC_rank', as_index = False).agg(count=('CustomerID', 'count'),
                                     total_profit=('Profit', 'sum')
)
abc_summary.head()
```

```
# Draw charts
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20,10))
colors = ['#2ecc71', '#f1c40f', '#e74c3c']

# Chart 1: Proportion of customers
ax1.pie(abc_summary['count'], labels=abc_summary['ABC_rank'], autopct='%1.1f%%', colors=colors, startangle=140, textprops={'for
ax1.set_title('Proportion of Customers', fontsize=16)

# Chart 2: Total profit contribution
sns.barplot(data=abc_summary, x='ABC_rank', y='total_profit', palette=colors, hue='ABC_rank', legend=False, ax=ax2)
ax2.set_title('Total Profit Contributed', fontsize=16)
ax2.set_xlabel('Rank', fontsize=14)
ax2.set_ylabel('', fontsize=14)
ax2.tick_params(labelsize=12)

plt.tight_layout()
plt.show()
```

```
# Compare Average Income
avg_income = customer_analysis.groupby('ABC_rank')['AnnualIncome'].mean()

# Initialize plot area with 1 row, 2 columns
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
colors = ['#2ecc71', '#f1c40f', '#e74c3c'] # Standard colors for A, B, C

# --- Chart 1: Average Income ---
sns.barplot(x=avg_income.index, y=avg_income.values, palette=colors, ax=axes[0], hue=avg_income.index, legend=False)
axes[0].set_title('Comparison of Average Income Across Groups', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Annual Income ($)')
axes[0].set_xlabel('Customer Group')

# Add data labels on top of bars
for i, v in enumerate(avg_income.values):
    axes[0].text(i, v + 500, f'${v:,.0f}', ha='center', fontweight='bold')

# --- Chart 2: Income Distribution (Boxplot) ---
# This chart helps to see the income range from lowest to highest for each group
sns.boxplot(data=customer_analysis, x='ABC_rank', y='AnnualIncome', palette=colors, ax=axes[1], hue='ABC_rank', legend = False)
axes[1].set_title('Income Distribution by Group', fontsize=14, fontweight='bold')
axes[1].set_ylabel('Income Range ($)')
```

```
axes[1].set_xlabel('Customer Group')
```

```
plt.tight_layout()
plt.show()
```

```
fig, axes = plt.subplots(1, 3, figsize=(20, 7))
```

```
# Choose colors
```

```
colormap_names = {
    'A': 'Greens_r',
    'B': 'YlOrBr_r',
    'C': 'Reds_r'
}
```

```
for i, rank in enumerate(['A', 'B', 'C']):
```

```
    # Get Top 5 occupations for each group, sorted in descending order by count
```

```
    top_data = customer_analysis[customer_analysis['ABC_rank'] == rank]['Occupation'].value_counts().head(5).reset_index()
    top_data.columns = ['Occupation', 'Count']
```

```
    cmap = plt.colormaps[colormap_names[rank]]
```

```
    bar_colors = [cmap(x) for x in np.linspace(0.1, 0.9, len(top_data))]
```

```
    # Draw horizontal bar chart for each group
```

```
    sns.barplot(data=top_data, x='Count', y='Occupation', palette=bar_colors, ax=axes[i], hue='Occupation', legend=False)
```

```
    axes[i].set_title(f'Top Occupations - Group {rank}', fontsize=15, fontweight='bold')
```

```
    axes[i].set_xlabel('Number of Customers')
```

```
    axes[i].set_ylabel('')
```

```
plt.tight_layout()
```

```
plt.show()
```

✓ IV. Conclusion

✓ 4.1. Problem 1

What percentage of the total customer base does Group A represent?

Group A accounts for only 12.9% of the total customer base (2,254 out of 17,415 customers).

This result adheres to and even concentrates more than the Pareto principle (80/20). Typically, this rule states that 20% of customers generate 80% of profit, but here, only ~13% of customers bring in 80% of the profit. Conclusion: This e-commerce platform has an extremely high-quality customer segment. Fluctuations within this group will directly and strongly affect the company's survival.

✓ 4.2. Problem 2

If the company only has 500 special gifts, does the list of Group A customers exceed this number?

The number of Group A customers (2254 people) already exceeds the available number of gifts (500 items).

Therefore, it is not possible to give gifts to all Group A customers.

✓ 4.3. Problem 3

Based on income and occupation information, what suggestions can be made for upcoming marketing campaigns to attract more new "Group A" customers?

Based on income and occupation data, it can be seen that, paradoxically, Group C customers (who buy less) have a higher average income (USD 57,378) than Group A (USD 56,681).

The occupations Professional and Skilled Manual are those contributing the most to the classified groups.

Group A is not the wealthiest group (in terms of average income) but spends the most, proving that they buy for practical value, convenience, or brand loyalty, rather than for show-off.

The e-commerce platform still has thousands of Professional customers in Group C. While this group also ranks high in Group A and B, they still top Group C in terms of customer count.

Suggestions for Marketing Strategy:

- Focus advertising on Professional and Skilled Manual occupational groups.
- Use channels like LinkedIn or professional forums to reach Professional and Skilled Manual groups.
- Focus on messages about: Durable quality, Professional customer service, Exclusive privileges for professionals.
- Upsell campaigns specifically for Professional occupational groups in Group C.

✓ V. Proposed Actions

- It is advisable to further filter Group A to find the Top 500 customers by profit (Group A+) or combine with criteria such as purchase frequency (RFM analysis) to ensure gifts reach the most important people.
- The upcoming CRM strategy should shift from "Mass Marketing" to "Personalized Marketing." Allocate 80% of resources to protect and care for the 12.9% of Group A customers, while running campaigns for occupational privileges to attract wealthy customers from Group C.